

Samuel Oliveira

Ph0enix



Github



LinkedIn



Website



EyeWeb Technical Report

Document Information:

Title	EyeWeb: Data Breach Verification Platform
Client	ISTEC Porto: Instituto Superior de Tecnologias Avançadas do Porto
Author	Samuel Oliveira
Date	April 2026
Version	1.0
Classification	Public

Version Control:

Version	Date	Author	Description
1.0	Abril - 2026	Samuel Oliveira	Final version of the report

Distribution:

Name	Position	Organization
José Oliveira	Developer	ISTEC Porto
Ana Rita Monteiro	Developer	ISTEC Porto
Vanina Kollen	Developer	ISTEC Porto
Tiago Carvalho	Developer	ISTEC Porto
Francisco Ribeiro	Systems Analyst	ISTEC Porto



Index

Introduction.....	5
Scope and Authorization.....	6
Analysis Methodology.....	7
General Architecture	7
Privacy Model, K-Anonymity.....	8
API Endpoints	9
Data Storage.....	10
Supabase Database (PostgreSQL)	11
Authentication System.....	12
Platform Security.....	13
Infrastructure and Deployment	15
Continuous Improvement Plan.....	16
Conclusion.....	17
References	18
Glossary	19



List of Figures

Figure 1: Overview of the EyeWeb architecture. 7

Figure 2: K-Anonymity privacy model flow. 8

Figure 3: Component communication during URL analysis. 10

Figure 4: Supabase entity-relationship diagram with RLS. 11

Figure 5: Email authentication and MFA (TOTP) for administrators. 12

Figure 6: Production infrastructure and service integration..... 15



Introduction

This report documents in detail the EyeWeb project, a cybersecurity platform developed as the Final Project of the 2nd year of the CTeSP in Cybersecurity at the Instituto Superior de Tecnologias Avançadas do Porto (ISTEC), academic year 2025/2026.

EyeWeb allows any user to check whether their personal data — email addresses, phone numbers, and passwords — have been exposed in known data breaches. Additionally, it offers intelligent analysis of potentially malicious URLs, identifying threats such as phishing, malware, and social engineering.

The main motivation behind the project lies in the growing need for accessible tools that allow ordinary citizens to protect their digital presence. The platform was designed with a total commitment to user privacy, using the K-Anonymity mathematical model to ensure that no personal data is exposed to the server during checks.

EyeWeb is publicly available at <https://eyeweb.vercel.app> and its source code is open, hosted at <https://github.com/Sam-Ciber-Dev/eyeweb>.



Scope and Authorization

This project was developed within the scope of the Final Project course of the CTeSP in Cybersecurity at ISTECS Porto, under academic supervision. The project was also presented at the Joteca, an academic organization of ISTECS, before representatives of companies and other higher education institutions.

The scope of the project covers:

- Verification of emails and phone numbers against public data breach databases, using SHA-256 hashing and the K-Anonymity model;
- Verification of compromised passwords using the same privacy model;
- URL security analysis using the Google Safe Browsing API and artificial intelligence (Groq LLaMA 3.3 70B);
- AI-integrated assistant chatbot to answer cybersecurity questions;
- Administration panel with multi-factor authentication (TOTP), real-time traffic monitoring, threat detection, and blocked user management;
- Secure authentication system with email code verification and Google OAuth integration.

All breach data used comes from public and freely accessible databases. No real personal user data is stored by the server — only cryptographic hashes are transmitted.



Analysis Methodology

The EyeWeb architecture follows the client-server model, with a clear separation between frontend and backend, communicating exclusively via REST API over HTTPS.

General Architecture

The project is composed of three main layers:

1. Frontend

Web application built with Next.js 16 (App Router), TypeScript and React, hosted on Vercel (CDG1 region — Paris). Responsible for the user interface, local SHA-256 hash computation, and local comparison of K-Anonymity results.

2. Backend

REST API built with FastAPI (Python 3.11+), hosted on Render.com (Frankfurt region). Responsible for business logic, dataset queries, integration with external APIs (Google Safe Browsing, Groq AI, URLScan.io), and traffic management.

3. Database

Supabase (managed PostgreSQL) for authentication, user profiles, verified URL cache, traffic logs, and admin chat. Hugging Face Datasets for storing breach and password datasets in Apache Parquet format, partitioned by hash prefix.

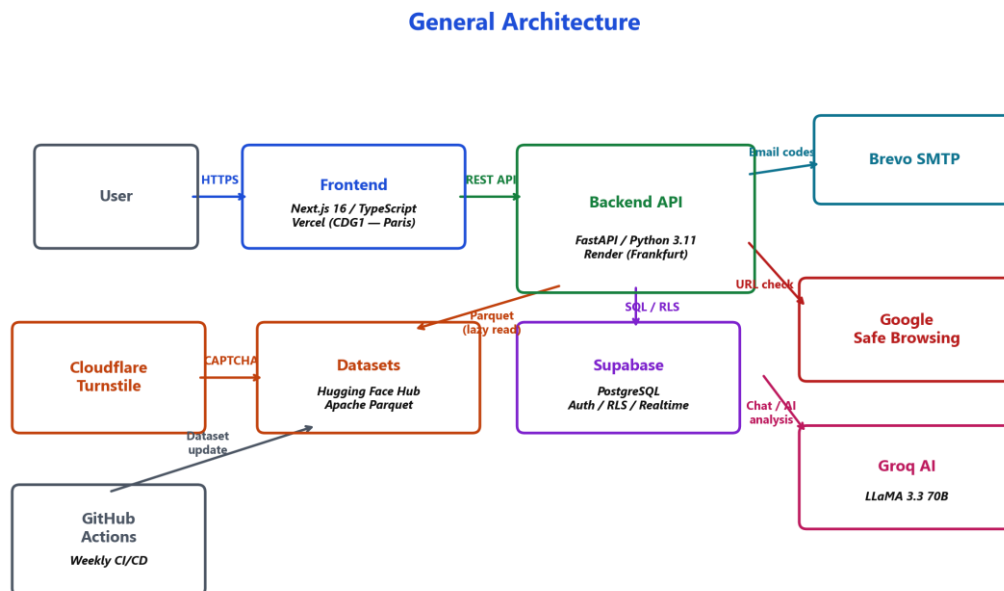


Figure 1: Overview of the EyeWeb architecture.



Privacy Model, K-Anonymity

K-Anonymity is the central pillar of EyeWeb's privacy architecture. The flow works as follows:

1. The user enters their email, phone number, or password in the browser;
2. The browser computes the SHA-256 hash of the data locally, without sending it to the server;
3. Only the first 5 characters of the hash (the prefix) are extracted;
4. The prefix is sent to the backend via HTTPS;
5. The backend returns ALL stored hashes that start with that prefix;
6. The browser locally compares whether the user's full hash is in the candidate list;
7. The server never knows which specific record was checked.

This approach guarantees mathematical privacy: even if network traffic were intercepted, the attacker would only know the 5-character prefix, which corresponds to hundreds or thousands of different records.

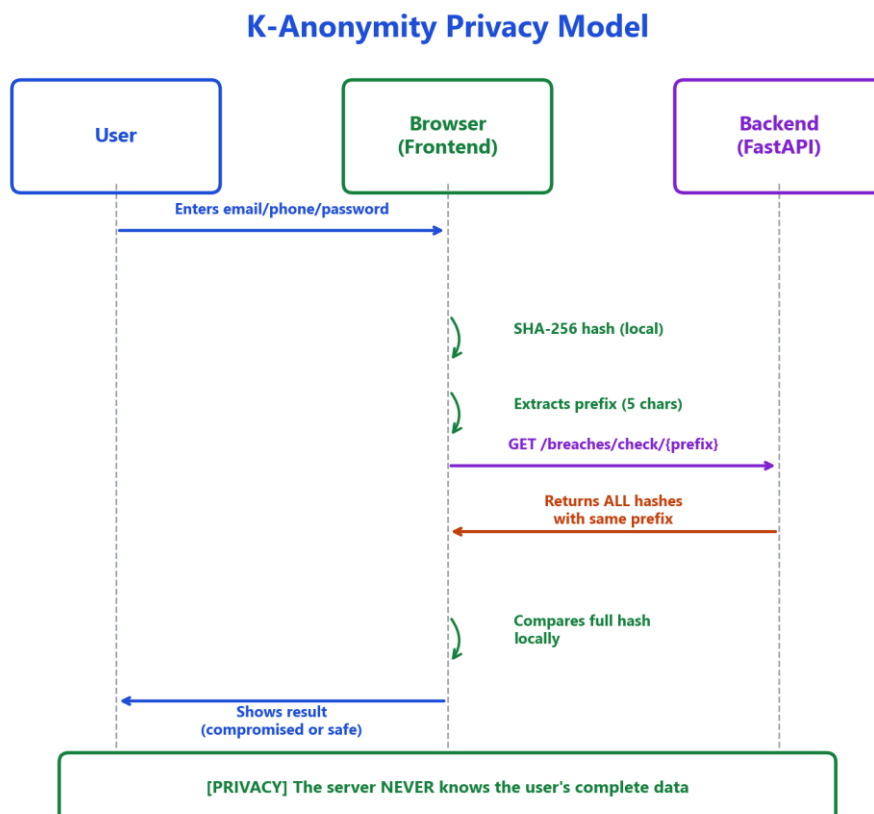


Figure 2: K-Anonymity privacy model flow.



API Endpoints

The API is organized into the following endpoint groups:

GET /api/v1/breaches/check/{prefix}

Checks emails and phone numbers against the breach dataset. Accepts a hexadecimal prefix of 2 to 6 characters and returns all matching candidates, including breach name, date, and types of exposed data.

GET /api/v1/passwords/check/{prefix}

Checks compromised passwords. Works identically to the breaches endpoint but queries the separate password dataset. Returns candidate hashes with occurrence frequency.

POST /api/v1/urls/check

Analyzes the security of a URL. Receives the full URL, normalizes it (blocks dangerous protocols such as javascript:, data:, file:; blocks private IPs; adds HTTPS if necessary), and performs checks with Google Safe Browsing and AI analysis. Implements stale-while-revalidate cache: fresh results (< 1 hour) are returned immediately; stale results (1h-30 days) are returned from cache while a re-check is initiated in the background; uncached URLs are checked in real time.

POST /auth/send-code

Generates and sends a verification code by email via Brevo SMTP. The session expires in 5 minutes.

POST /auth/verify-code

Validates the code selected by the user. On success, the frontend obtains a JWT token from Supabase for authenticated access.

POST /admin/verify-mfa

Verifies a 6-digit TOTP code for administration panel access. Uses HMAC-SHA256 with a 30-second interval and a 2-minute tolerance window. Two failed attempts result in a 72-hour block by IP and device fingerprint.

POST /user/chat/chat

Public chatbot integrated with Groq LLaMA 3.3 70B. Includes protections against prompt injection (regex for XSS, SQL injection, jailbreak) and output validation (checks if the response contains keywords relevant to EyeWeb).

GET /admin/traffic/*

Set of protected endpoints for traffic monitoring: dashboard statistics, real-time active connections, paginated logs, suspicious activity, and blocked IP/device management.



API Endpoints

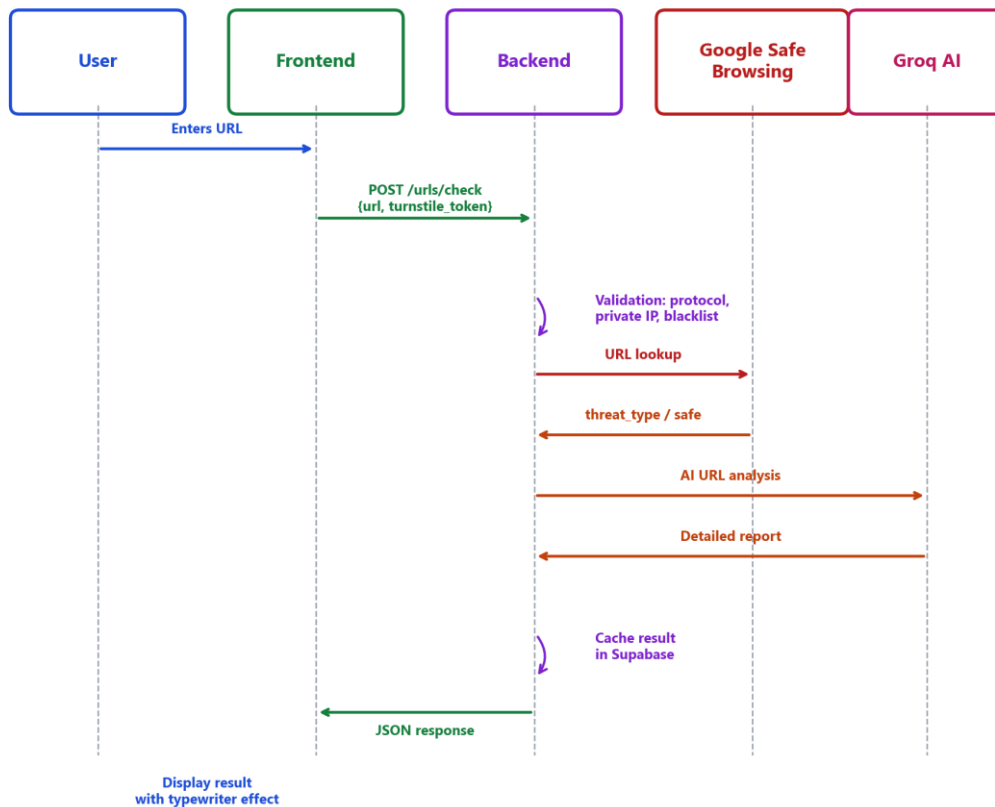


Figure 3: Component communication during URL analysis.

Data Storage

The breach and password datasets are stored on Hugging Face Datasets in Apache Parquet format, partitioned into 256 files (00.parquet through ff.parquet) by the 2-character prefix of the SHA-256 hash. This strategy enables:

Lazy reading

Only the required partition is downloaded, optimizing memory usage on Render's free tier (512 MB RAM);

LRU cache with TTL

Up to 100 partitions are kept in memory for 1 hour, avoiding repeated downloads;

Scalability

New records are added via a GitHub Actions workflow that runs weekly, downloading new data and updating the Parquet files in the Hugging Face repository.



Supabase Database (PostgreSQL)

Supabase serves as the relational database for operational data:

- **Profiles:** User data (id, email, role, mfa_secret);
- **verification_codes:** Temporary authentication codes (session_id, correct_code, expires_at); **correct_code**
- **url_scans:** Verified URL cache (url_hash, original_url, status, ai_opinion, threat_details, last_check);
- **traffic_logs:** Visit logs with geolocation, VPN detection, and device fingerprint (30-day retention, automatic cleanup at midnight UTC);
- **blocked_ips / blocked_devices:** Manual or automatic block records;
- **admin_chat_messages:** Admin chat history with Supabase Realtime support.

All sensitive tables have Row-Level Security (RLS) enabled, ensuring that regular users only access their own data.

Supabase Database (PostgreSQL)

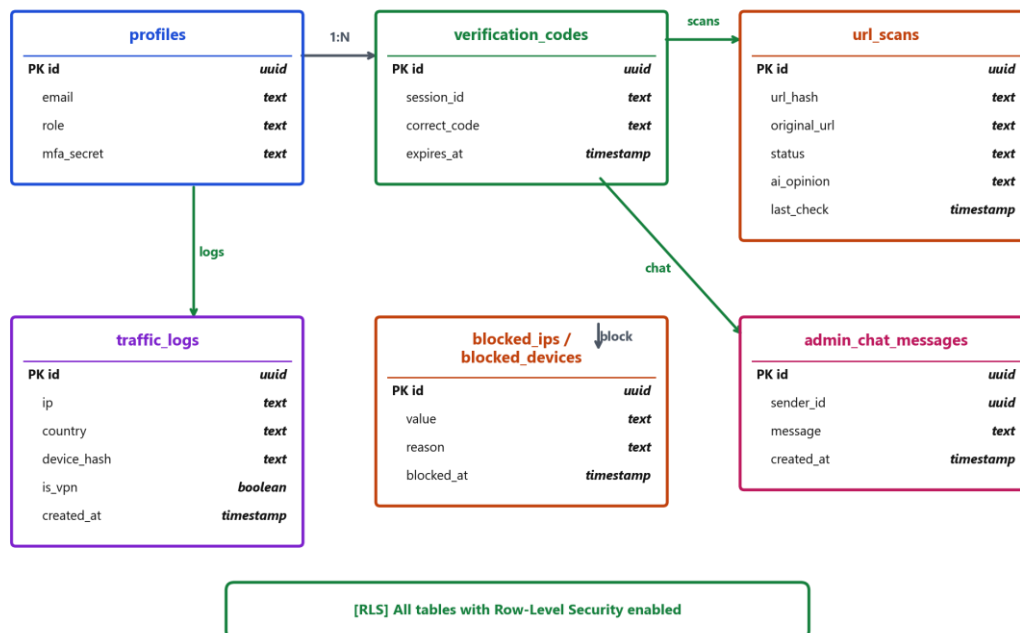


Figure 4: Supabase entity-relationship diagram with RLS.



Authentication System

EyeWeb's authentication combines multiple mechanisms:

Email code login

The user receives 3 codes by email (via Brevo SMTP), with only one being valid. This anti-phishing approach makes code reuse attacks harder.

Google OAuth

Login via Google account, with automatic profile registration.

MFA for administrators

6-digit TOTP codes generated offline by a Python desktop application (eyeweb_auth.py). Codes never travel over the network; verification is done by HMAC-SHA256 comparison on the server.

JWT Tokens

After authentication, Supabase issues JWT tokens that are validated on each protected request. The backend verifies the token with Supabase and compares the SHA-256 hash of the email with the ADMIN_EMAIL_HASH environment variable to confirm administrative access.

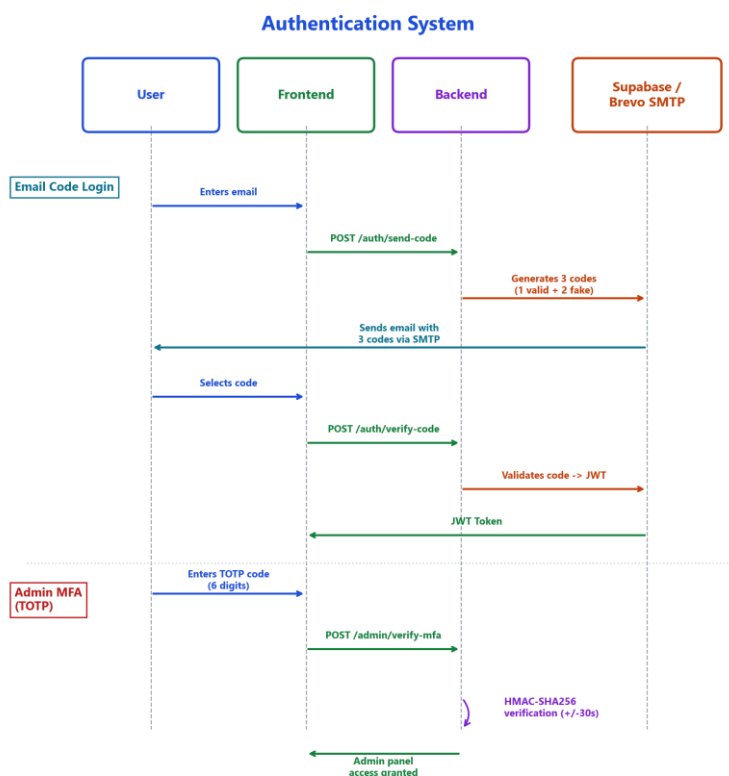


Figure 5: Email authentication and MFA (TOTP) for administrators.



Platform Security

The following security measures are implemented:

- **Restrictive CORS:** In production, only `https://eyeweb.vercel.app` is allowed; in development, `localhost` is added;
- **Swagger/Docs disabled in production:** The `/docs`, `/redoc`, and `/openapi.json` endpoints return 404 when `ENVIRONMENT=production`;
- **Rate limiting:** 60 requests per minute per IP on public endpoints;
- **Cloudflare Turnstile:** CAPTCHA on the registration/login form to prevent bots;
- **URL validation:** Blocking of malicious protocols (`javascript:`, `data:`, `vbscript:`, `file:`), private IPs (`127.x`, `192.168.x`, `10.x`, `172.16-31.x`), and malformed protocols (`ht+ps`, `https`);
- **Chatbot prompt injection protection:** Regex to detect XSS, SQL injection, jailbreak, DAN mode, and other manipulation attempts. Dual validation: user input AND AI output;
- **Security headers (Vercel):** `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `X-XSS-Protection: 1; mode=block`;
- **Device fingerprinting:** Canvas, WebGL, Audio API, screen resolution, CPU, RAM, timezone, platform. Hardware-only hash to detect the same device across different browsers. Score $\geq 70/100$ = same device;
- **Real-time threat detection:** Automatic identification of scanners (Nmap, SQLMap, Nikto, Burp Suite), SQL injection attempts, path traversal, and access to suspicious paths (`/wp-admin`, `/.env`, `/phpmyadmin`);
- **Automatic and manual blocking:** IPs and devices can be blocked automatically for suspicious behavior or manually by the administrator;
- **Docker containerization:** The backend runs as a non-root user (`appuser`) inside a Python 3.11-slim container, without pip cache.
- **Frontend User Interface:** The frontend is a Next.js 16 application with App Router, written in TypeScript and React. The main features include:
- **Four integrated checkers:** Email, Phone, Password, and URL, organized in tabs;
- **Local hash computation:** All SHA-256 cryptography is executed in the user's browser, ensuring raw data never leaves the device;
- **Password strength meter:** Real-time assessment based on length, character diversity, and anti-patterns (repetitions, letters only);
- **Typewriter effect in AI analysis:** URL analysis results are presented with a typing animation, creating an interactive experience;
- **Eye Chatbot:** Floating widget that allows users to ask cybersecurity questions in Portuguese or English;
- **Full internationalization (PT/EN):** Over 300 translation keys, with automatic browser language detection;



- **Administration panel:** Dashboard with traffic charts (Recharts), country map, event timeline (Wireshark-style), VPN detection, and real-time admin chat (Supabase Realtime);
- **Visit tracking:** Invisible component that logs visits and sends heartbeats every 5 minutes with device fingerprint;
- **Blocking middleware:** Before each page loads, checks whether the visitor's IP or device fingerprint is blocked;
- **Framer Motion animations:** Smooth transitions between components and pages;
- **Legal pages:** Privacy Policy and Terms of Service, accessible from the footer.



Infrastructure and Deployment

Frontend: Hosted on Vercel, CDG1 region (Paris), with auto-deploy from the main branch of the GitHub repository. Static + SSR Next.js build;

Backend: Hosted on Render.com, Frankfurt region, with auto-deploy via Docker. Free tier plan (512 MB RAM, spin-down after inactivity);

Database: Supabase free tier, with authentication, PostgreSQL, Realtime, and Storage;

Datasets: Hugging Face Hub, private repositories with read token;

CI/CD: GitHub Actions: weekly workflow for breach dataset update (actions/checkout@v6, actions/setup-python@v6);

SMTP: Brevo (smtp-relay.brevo.com:587) for sending verification codes;

DNS: Domain managed by Vercel (eyeweb.vercel.app).

Infrastructure & Deployment

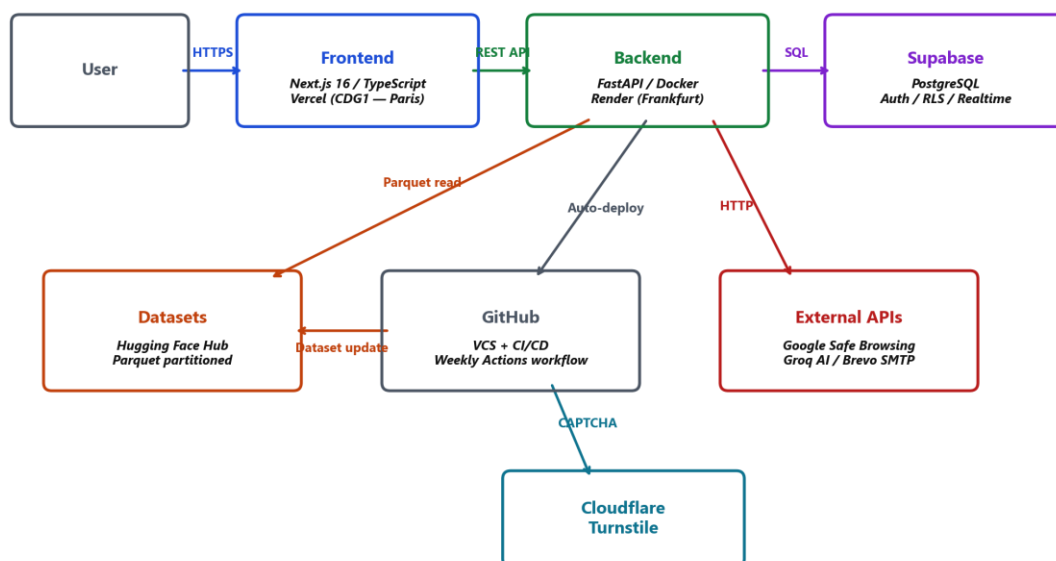


Figure 6: Production infrastructure and service integration.



Continuous Improvement Plan

Based on the technical analysis performed, the following is recommended for the project's future evolution:

- 1. Migration to paid plans:** Render's free tier results in spin-down after inactivity, causing cold starts of ~30 seconds. A paid plan would eliminate this latency and allow more RAM for partition cache.
- 2. Enabling compromised password protection in Supabase:** This feature is only available on Supabase's Pro plan. However, EyeWeb already implements its own password verification via K-Anonymity, which partially mitigates this limitation.
- 3. Adding more breach data sources:** Integrate additional databases (e.g., Have I Been Pwned API, BreachDirectory) to increase detection coverage.
- 4. Notification implementation:** Allow registered users to receive automatic alerts when their data appears in new breaches.
- 5. Automated testing:** Implement unit and integration test suites with pytest (backend) and Jest/Testing Library (frontend) to ensure stability in future updates.
- 6. External security audit:** Submit the platform to a formal pentest by an external entity to identify vulnerabilities not detected in the internal audit.
- 7. Custom SSL certificate:** Although Vercel and Render provide automatic HTTPS, a dedicated certificate would strengthen user trust.
- 8. Advanced monitoring:** Integrate tools such as Sentry (error tracking) and UptimeRobot (availability) for continuous production monitoring.



Conclusion

EyeWeb fulfills the objectives proposed as the Practical Application Project of the CTeSP in Cybersecurity: a functional, secure, and accessible platform that allows any user to check whether their personal data has been compromised in known data breaches.

The pillars that support the project are:

- **Privacy by design:** The K-Anonymity model ensures that no personal user data is exposed to the server. All hash comparison is performed locally in the browser, making the platform mathematically private.
- **Modern architecture:** The separation between frontend (Next.js/TypeScript on Vercel) and backend (FastAPI/Python on Render) enables independent scalability, simplified maintenance, and continuous deployment.
- **Layered security:** From input validation and CORS protection, through real-time threat detection and device fingerprinting, to automatic attacker blocking and multi-factor authentication for administrators.
- **Integrated artificial intelligence:** URL analysis with Groq LLaMA 3.3 70B and the assistant chatbot demonstrate the practical application of AI in cybersecurity, with prompt injection protections implemented.

The project demonstrates that it is possible to build robust and private cybersecurity tools using exclusively free services (Vercel, Render, Supabase, Hugging Face, Groq), making them accessible for academic and educational purposes.

The presentation of EyeWeb at the ISTEC Porto Joteca validated the project's relevance before representatives of companies and higher education institutions, confirming its value as a tool for raising awareness about personal data security.



References

- **FastAPI:** <https://fastapi.tiangolo.com/>
- **Next.js:** <https://nextjs.org/>
- **Supabase:** <https://supabase.com/>
- **Hugging Face Datasets:** <https://huggingface.co/docs/datasets/>
- **Groq:** <https://groq.com/>
- **Google Safe Browsing API:** <https://developers.google.com/safe-browsing/>
- **K-Anonymity:** <https://dataprivacylab.org/dataprivacy/projects/kanonymity/>
- **Have I Been Pwned:** <https://haveibeenpwned.com/>
- **Cloudflare Turnstile:** <https://developers.cloudflare.com/turnstile/>
- **Apache Parquet:** <https://parquet.apache.org/>
- **Render:** <https://render.com/>
- **Vercel:** <https://vercel.com/>
- **Brevo (Sendinblue) SMTP:** <https://www.brevo.com/>
- **Docker:** <https://www.docker.com/>
- **OWASP Top 10:** <https://owasp.org/www-project-top-ten/>



Glossary

- **API (Application Programming Interface):** Interface that enables communication between different software systems.
- **CORS (Cross-Origin Resource Sharing):** Security mechanism that restricts HTTP requests between different domains.
- **CSRF (Cross-Site Request Forgery):** Attack that forces an authenticated user to perform unwanted actions.
- **CTeSP:** Higher Professional Technical Course, level 5 qualification.
- **Data Breach:** Security incident in which personal data is accessed, disclosed, or stolen without authorization.
- **Device Fingerprint:** Unique identifier of a device based on its hardware and software characteristics.
- **FastAPI:** Modern web framework for Python, optimized for high-performance RESTful APIs.
- **Hash (SHA-256):** Cryptographic function that transforms data of any size into a fixed 256-bit value, irreversible.
- **HMAC:** Hash-based Message Authentication Code, hash-based message authentication mechanism.
- **JWT (JSON Web Token):** Compact token format for secure information transmission between parties.
- **K-Anonymity:** Privacy model that ensures each record is indistinguishable from at least K-1 other records.
- **LLaMA:** Large Language Model Meta AI, large-scale language model developed by Meta.
- **MFA (Multi-Factor Authentication):** Authentication that requires two or more verification factors.
- **Next.js:** React framework for web applications with server-side rendering and static generation.
- **Parquet:** Columnar storage format optimized for large data volumes.
- **PAP:** Practical Application Project, final project of the CTeSP.
- **RLS (Row-Level Security):** Row-level security policy that restricts access to specific database records.
- **SMTP (Simple Mail Transfer Protocol):** Standard protocol for sending emails.
- **TOTP (Time-based One Time Password):** Algorithm that generates time-based disposable passwords.
- **TypeScript:** JavaScript superset with static typing, used in the frontend.
- **XSS (Cross-Site Scripting):** Attack that injects malicious scripts into web pages.

Thank You

Ph0enix



Github



LinkedIn



Website